

5. Remote Procedure Call

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2024

5.1 Vorschau

- Grundlagen
- Fehler
- Web Services
- REST
- gRPC
- Zusammenfassung



5.2 Grundlagen



Motivation

- Remote Procedure Call (RPC) als Oberbegriff = Fernaufruf
 - Ursprünglich nur Funktionen ohne Objekte
 - Heute objekt-orientiert und somit entfernte Methodenaufrufe
- Ziel: Transparenz
 - Ein Fernaufruf soll so aussehen wie ein lokaler Funktionsaufruf
 - Automatisches Übertragen der Parameter und des Ergebnisses
 - Möglichst sprach- und plattformunabhängig
 - Somit einfacher, als Socket-Programmierung



Motivation

- Fernaufrufe werden häufig verwendet
 - Google gRPC
 - Facebook / Apache Thrift
 - Twitter Finagle
 - ...
- Ursprüngliche Idee von 1984
 - Andrew D. Birrell and Bruce Jay Nelson, „Implementing remote procedure calls“, ACM Transactions on Computer Systems, 2(1):39–59, 1985.

Herausforderungen

- Zusätzliche Fehlerquellen (gegenüber lokalem Aufruf):
 - Was passiert wenn eine Netzwerknachricht verloren geht?
 - Was ist, wenn der Client, Server oder das Netzwerk ausfallen?
- Geschwindigkeit:
 - Ein Fernaufruf ist langsamer wie ein lokaler Aufruf
 - Lokal: $<10\text{ns}$
 - Entfernt:
 - $<10\text{-}1000\mu\text{s}$ im LAN (in einem Data-Center)
 - $<10\text{-}100\text{ms}$ im WAN (im Internet)

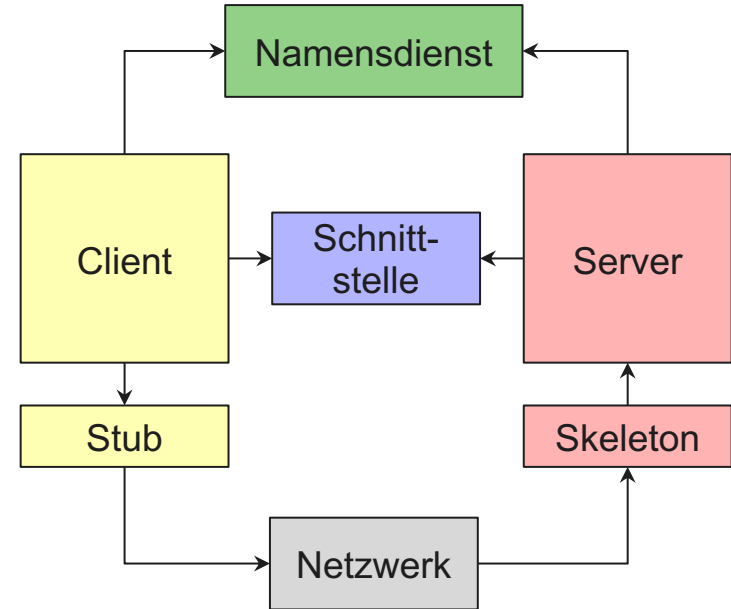


Herausforderungen (2)

- Heterogenität
 - Client und Server verwenden u.U. unterschiedliche Programmiersprachen
 - Datentypen haben daher unterschiedliche Byte-Größen
 - Unterschiedliches Alignment von Daten (4 vs 8 Byte)
 - Eventuell andere Byte-Order (Big vs Little Endian)

Übersicht der Fernaufruf-Komponenten

- Schnittstelle: sprach- und plattformunabhängige Beschreibung
- Server
 - Bietet fernaufrufbare Funktionen an
- Client
 - Ruft Funktionen auf dem Server auf
- Stub/Skeleton
 - Automatisch generierter Code, welcher die Kommunikation für die Fernaufrufe realisiert
- Namensdienst (optional)
 - Server macht Dienst bekannt
 - Client kann Dienste dort finden

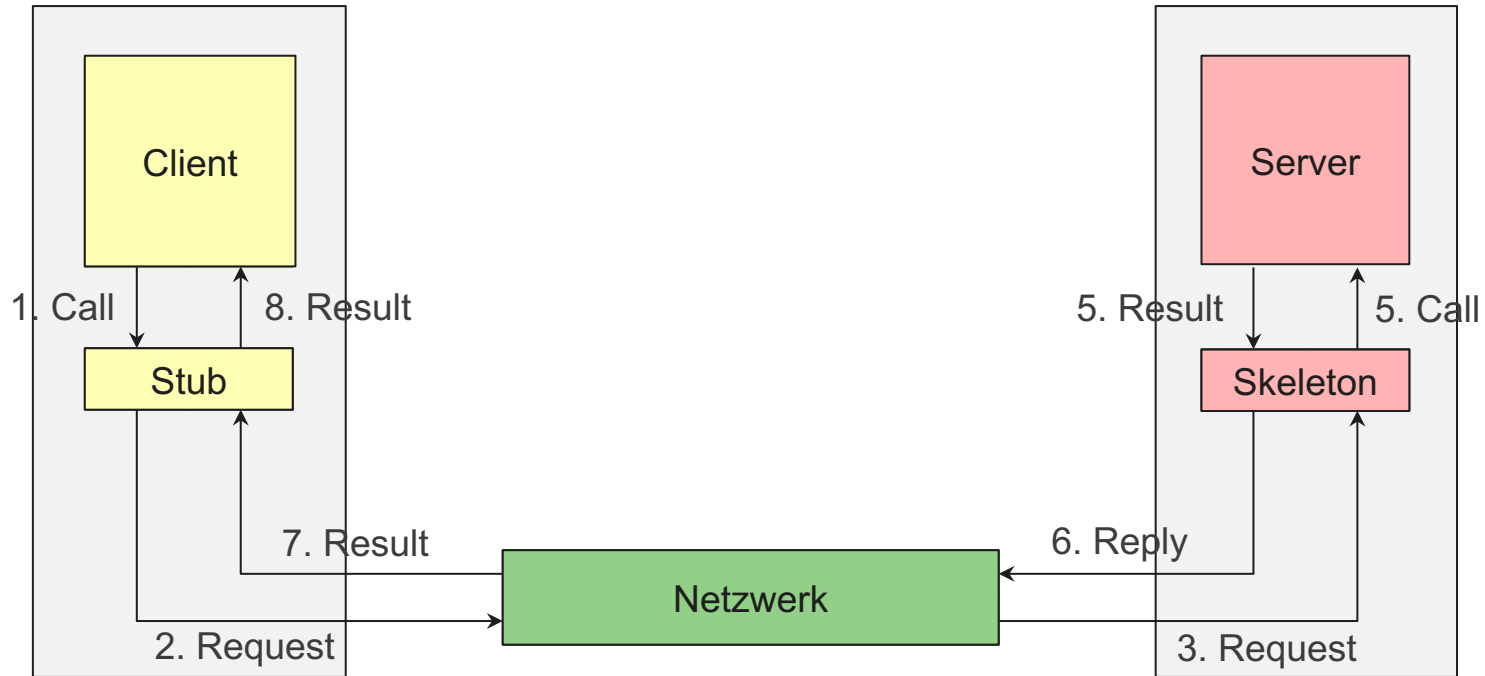


Stub und Skeleton

- Stub (Client)
 - Stellvertreter des Servers auf Client-Seite
 - Umsetzung des Aufrufs in einen Nachrichtenaustausch
 - Verpacken der Aufrufparameter und Entpacken des Rückgabewerts
- Skeleton (Server)
 - Stellvertreter des Clients auf Server-Seite
 - Umsetzung der Aufrufrückkehr in einen Nachrichtenaustausch
 - Entpacken der Aufrufparameter und Verpacken des Rückgabewerts
- Oft gibt es auch andere Namen, beispielsweise Proxy, Stellvertreter, ...



Stub und Skeleton



(Un)-Marshalling der Daten

- = Basis für eine Interaktion per Nachrichtenaustausch
 - Marshalling: Serialisierung aller zu sendender Daten in eine Nachricht
 - Unmarshalling: Deserialisierung einer Nachricht nach Empfang
- Nachrichteninhalte bei Fernaufrufen
 - Anfrage
 - Bei Methodenfernaufruf: Bezeichner des entfernten Objekts
 - Bezeichner der aufzurufenden Prozedur / Methode
 - Parameter
 - Antwort
 - Normalfall: Rückgabewert(e)
 - Fehlerfall: Informationen zur Fehlersituation (z. B. Exception)



Parameter: Arten

- Eingabeparameter
 - Informationsfluss: Client $\rightarrow$ Server
 - Bestandteil der Anfrage (Call-by-Value)
- Ausgabeparameter
 - Informationsfluss: Client $\leftarrow$ Server
 - Bestandteil der Antwort (Call-by-Result)
- Ein-/Ausgabeparameter
 - Informationsfluss: Client $\longleftrightarrow$ Server
 - Bestandteil beider Nachrichten (Call-by-Value-Result)
 - Ersetzen des Werts auf Client-Seite durch den Wert in der Antwort
- Abhängig von der Programmiersprache (teilweise keine Unterscheidung)



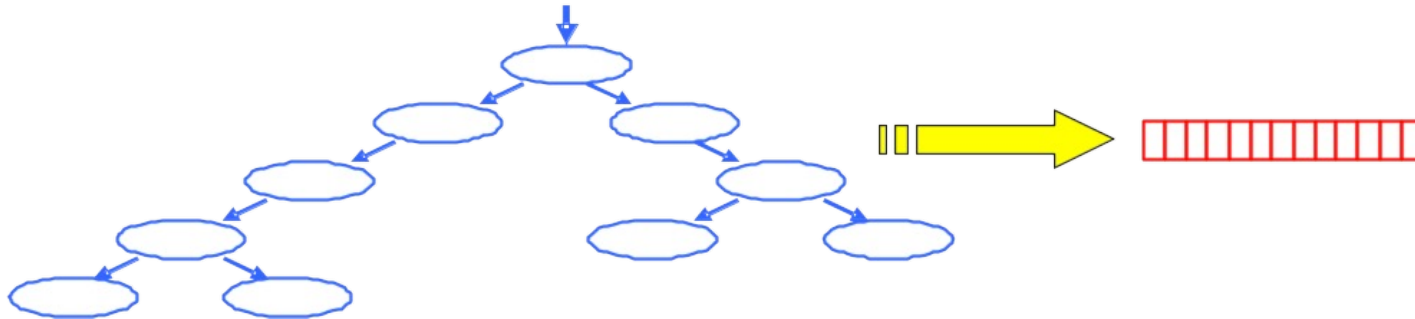
Parameter: Referenzen

- Problem: Umsetzung von Call-by-Reference
 - Client und Server haben keinen gemeinsamen Speicher
 - Versand von Zeigern in Anfragen / Antworten geht nicht!
- Lösung 1: Übermittlung einer Kopie des referenzierten Objekts/Struct
 - Problem ein Objekt/Struct kann wiederum eine Referenz beinhalten usw.
 - Daher müssen alle erreichbaren Objekte/Structs übertragen werden
- Lösung 2: Realisierung mittels Remote-Referenz
 - Übermittlung eines systemweit eindeutigen Zeigers anstatt der Speicheradresse
 - Dann erfolgt der Zugriff auf den Parameter wiederum per Fernaufruf



Parameter: Referenzen (2)

- Werden Kopien für eine Referenz übertragen, so müssen auch Referenzen in den Objekten beachtet werden
- Verzeigte Strukturen werden serialisiert ("flachgeklopft") und komplett übertragen → transitive Hüllenbildung
- Wird beispielsweise die Referenz auf die Wurzel eines Binärbaumes übergeben, so wird der gesamte Baum übertragen (u.U. sehr aufwändig)
→



Automatische Generierung von Stubs und Skeletons

- Durch den Compiler oder zusätzliche Werkzeuge
- Variante 1: Annotation von fernaufrufbaren Methoden
 - Beispiel: WebServices
- Variante 2: explizite Schnittstellenbeschreibung in einer separaten Datei
 - Beispiel: Interface Definition Language (IDL)
 - C-Syntax für RPC
 - C++-Syntax für Corba
 - Mit zusätzlichen Attributen in, out (um Parameterübertragung zu optimieren)



5.3 Fehler



Gutartige Fehler

- Was kann alles schief gehen?
 - Client kann abstürzen und neustarten
 - Netzwerkpakete können verloren gehen
 - Server kann abstürzen und neustarten
 - Netzwerk und/oder Server können zeitweise sehr langsam sein
- Problem für den Client
 - Er kennt nicht die Fehlerursache
- Lösung: falls keine Antwort (nach einer bestimmten Zeit) ankommt, Fernaufruf wiederholen
 - Neues Problem: vielleicht wurde der Aufruf schon erfolgreich durchgeführt und nur die Antwort ging verloren

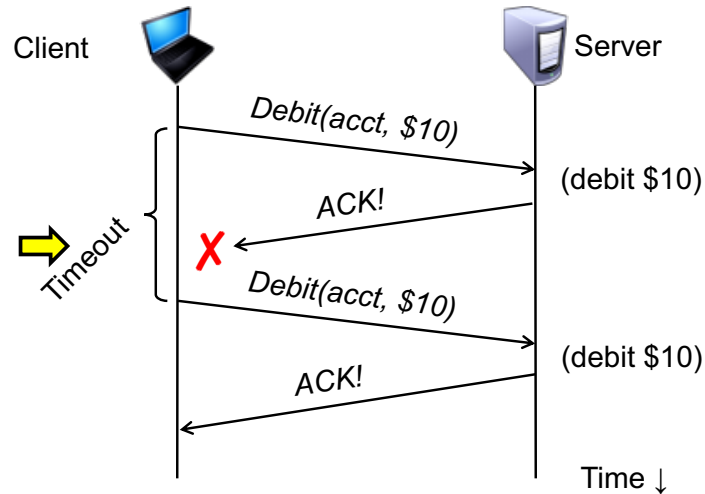


At-Least Once

- Semantik: Fernaufruf (mit evt. Seiteneffekten) wird auf dem Server mindestens ein Mal durchgeführt
- Client-Stub wartet bestimmte Zeit auf eine Antwort vom Server
- Falls in dieser Zeit keine Antwort vom Server eingeht, wird der Aufruf wiederholt
- Dies wird einige Male gemacht und falls es dann immer noch nicht klappt, wird an die Anwendung ein Fehler gemeldet

At-Least Once - Problem

- Gute geeignet, falls auf dem Server nur Daten gelesen werden
- Aber problematisch falls es Seiteneffekte auf dem Server gibt
 - Dann muss die Anwendung mit Duplikaten umgehen können
 - Hier würden 2x \$10 abgebucht



At-Most Once

- Semantik: Fernaufruf (mit evt. Seiteneffekten) wird auf dem Server höchstens ein Mal durchgeführt (u. U. kein einziges Mal)
- Server-Skeleton erkennt wiederholte Fernaufrufe und schickt direkt die alte Antwort, ohne den Aufruf erneut durchzuführen
- Damit wiederholte Fernaufrufe erkannt werden können müssen diese um eine global eindeutige ID erweitert werden

At-Most Once - Probleme

- Wie lange muss der Server alte Antworten und IDs speichern?
- Client kann dem Server mitteilen, dass er die Antwort bekommen hat
 - Optimierung: „piggyback“ mit nächstem Fernaufruf
 - Aufwändig, falls viele parallele Fernaufrufe stattfinden
- Was ist wenn der Server abstürzt?
- Die alten Antworten und IDs müssen vorsorglich auf Disk gespeichert werden, damit diese beim Neustart geladen werden können
 - Ansonsten würde der Server Duplikate nicht erkennen und Aufrufe doch mehrfach durchführen



Exactly Once

- Fernaufruf (mit evt. Seiteneffekten) wird auf dem Server genau ein Mal ausgeführt
- Benötigt:
 - Wiederholung von Fernaufrufen (wie bei At-Least Once)
 - Sowie die Filterung von Duplikaten (wie bei At-Most Once)
 - Zusätzlich muss der Client offene Fernaufrufe verwalten und auf Disk speichern, damit er bei einem Absturz diese erneut (mit der gleichen ID) wiederholen kann
 - Und auch der Serverzustand muss komplett gesichert und im Fehlerfall wiederhergestellt werden
- Sehr aufwändig und kaum verwendet in der Praxis



5.4 Web Service



Web-Services: Ziele

- Offener, plattform- bzw. sprachunabhängiger Standard, bei dem nur die Schnittstellen definiert werden
- Siehe „Web Services Architecture“, W3C, 2005.
- Ursprünglich auch Vision vom Markt der Web-Services:
 - Softwarefirmen verkaufen Web-Services-Software
 - Dienstanbieter bieten Web-Services an
 - Services laufen lokal oder auf Rechnern von Application-Service-Providers (ASP)



Web-Services: Komponenten

- **HTTP** oder **HTTPS** fungiert als Transportschicht
- **WSDL** als standardisierte Service-Beschreibung
 - Web Services Description Language
- **SOAP** als plattformunabhängige Protokollspezifikation
 - Simple Object Access Protocol
- **UDDI** als verteilter Namensdienst
 - Universal Description, Discovery and Integration
 - Wird nicht mehr genutzt



Web-Services: XML

- Basieren auf XML (extensible markup language):
- Auszeichnungssprache zur Darstellung **hierarchisch strukturierter** Daten in Form von Textdaten (z.B. ASCII, UTF-8)
- Unterstrukturen mit Start- `<Tag-Name>` und End-Tag `</Tag-Name>` oder einem Empty-Element-Tag `<Tag-Name/>`
- Attribute bei einem Tag (Attribut-Name = "Attribut-Wert") für Zusatzinformationen

XML-Beispiel

```
<books>
  <book>
    <author>Karl May</author>
    <title>Winnetou</title>
    <ISBN>3-7802-0170-4</ISBN>
    <price format="EUR"/>
  </book>
  <pubinfo>
    <town>Bamberg</town>
  </pubinfo>
  <publisher> KM-Verlag
</publisher>
</books>
```



Zusammenspiel der Komponenten

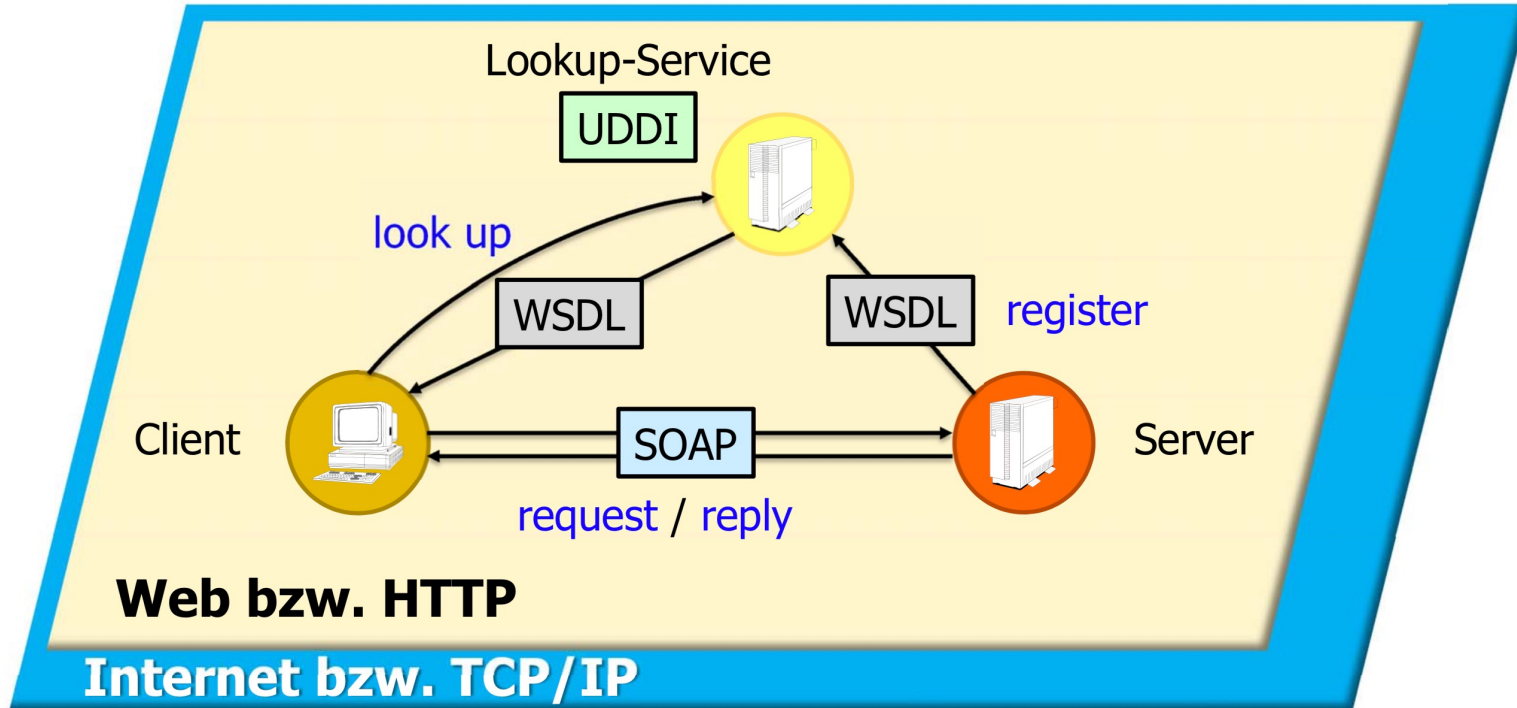


Bild von Prof. Mattern, ETH Zürich, Verteilte Systeme



Kommunikation

- Meist werden Methoden nur annotiert.
 - Z.B. @WebService in Java
- Die WSDL wird häufig dynamisch auf Anfrage generiert
- Mithilfe der WSDL werden wiederum automatisch die Stubs erzeugt

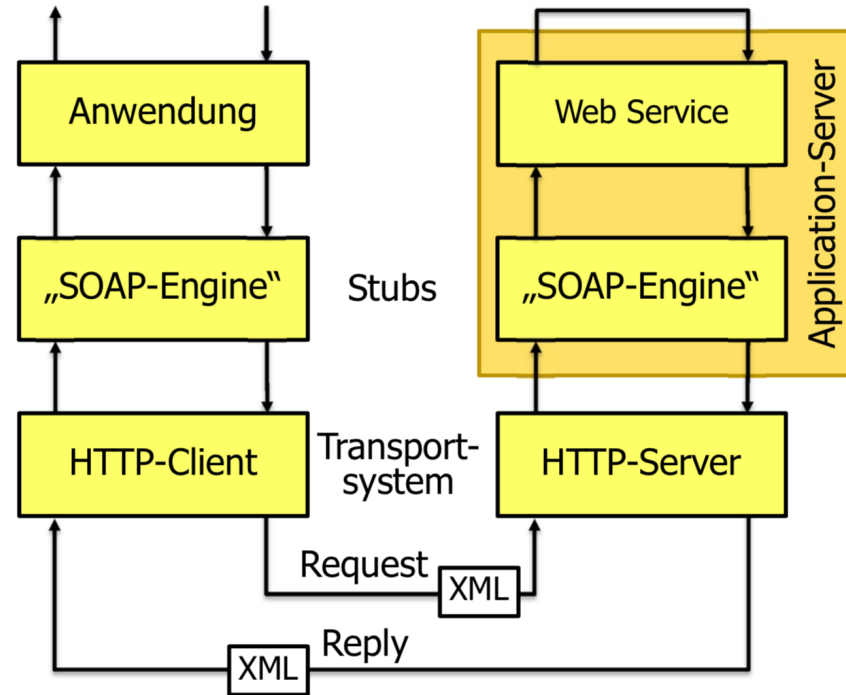


Bild von Prof. Mattern, ETH Zürich, Verteilte Systeme

5.5 REST



Representational State Transfer (REST)

- Intention: Entwicklung eines Regelwerks für hochskalierbare Systeme zur Bereitstellung von Anwendungen und Diensten über das Internet

„The name “**Representational State Transfer**” is intended to evoke an image of **how a well-designed Web application behaves**: a network of webpages (a virtual state-machine), where the user progresses through the application by selecting links (state transitions), resulting in the next page (representing the next state of the application) being transferred to the user [...]”

R. Fielding, „Architectural Styles and the Design of Network-based Software Architectures“, Dissertation, 2000.

- Architekturkonzept
 - Sammlung von Grundprinzipien, keine konkrete Implementierung
 - Anwendungsbeispiel: World Wide Web (WWW), siehe später



Zentrale Konzepte

1. Aufteilung in Client und Server
2. Zustandslose Interaktion
3. Ressourcen & Repräsentationen



1. Aufteilung in Client und Server

- **Server:** Bereitstellung eines Dienstes und Verwaltung der Daten
- **Client:** Nutzung von Diensten durch Senden von Anfragen an den Server
- Vorteile
 - Beide Seiten können unabhängig voneinander weiterentwickelt werden, solange die Schnittstelle zwischen ihnen stabil bleibt.
 - Erhöhte Portabilität der Client-Seite, auch in heterogenen Umgebungen
- Nachteile
 - Server-Seite kann bei schlechter Skalierbarkeit zum Flaschenhals werden



2. Zustandslose Interaktion

- Zustandslose Kommunikation über HTTP
 - Anfragen müssen den Zustand jeweils mitübertragen
 - Client muss alle Informationen (Kontext) mit einer Anfrage mitliefern, sodass der Server in „wiedererkennt“
- Vorteil
 - Einfachere parallele Bearbeitung von Anfragen → bessere Server-Skalierbarkeit
 - Einfache und schnelle Fehlererholung, falls der Server abstürzt
- Nachteil
 - Eventuell Mehraufwand durch vielfaches Senden von Kontextinformationen



3. Ressource & Repräsentation

- Identifikation von **Ressourcen** mithilfe Universal Resource Identifiers (URIs)
 - Beispiel `http://www.boeing.com/aircraft/747`
 - Ressource als abstraktes Konzept
 - Referenz ist nicht an die Existenz eines Objekts gebunden
 - Beispiel: Buch identifiziert über URI
- Ressourcen können unterschiedliche **Repräsentationen** haben
 - Beispiel: Repräsentation eines Buchs: PDF-Datei oder Taschenbuch



Beispiel für RESTful System: World Wide Web

- Zugriff auf Ressourcen (Web-Seiten, Bilder, Videos, ...)
 - Referenzierung von Ressourcen durch URIs
 - Repräsentationen ist konkretes Format (XML, gif, mp4 etc.)
 - Beziehungen/Zustandsübergänge durch Hyperlinks realisiert
- Einsatz von HTTP
 - Jede Nachricht enthält Host-Name & URI
 - HTTP als Anwendungsprotokoll
 - Einhaltung der spezifizierten Semantiken der HTTP-Requests
 - GET und HEAD sind rein lesende Methoden
 - POST, PUT und DELETE sind modifizierende Methoden
 - Antworten im Status-Code der HTTP-Antworten
 - 200: OK
 - 401: Unauthorized
 - 404: Not Found



Weitere Prinzipien um einen Server zu entlasten

- Transparente indirekte Kommunikation über einen Load-Balancer
 - Dieser verteilt dann die Requests auf mehrere Server-Replikate
- Code (z.B. JavaScript) auf dem Client herunterladen
- Zwischenspeicherung von Antwortnachrichten



SOAP vs REST

SOAP	REST
SOAP ist ein Protokoll	REST ist ein Architektur-Stil
Funktions-getrieben (Service-Schnittstelle beschrieben durch WSDL)	Daten-getrieben (fokussiert sich auf die Bereitstellung von Ressourcen über URIs)
Verwendet immer XML	Verschiedene Formate, u.a. JSON
Benötigt mehr Bandbreite	JSON ist kompakter
Komplexer zu Programmieren	Einfacher zu verwenden
Antworten können nicht gecacht werden, da SOAP immer POST-Anfragen schickt, welche in HTTP als nicht-idempotent betrachtet werden	Antworten können gecacht werden

- REST ist ein Industrie-Standard für Web APIs



5.6 gRPC

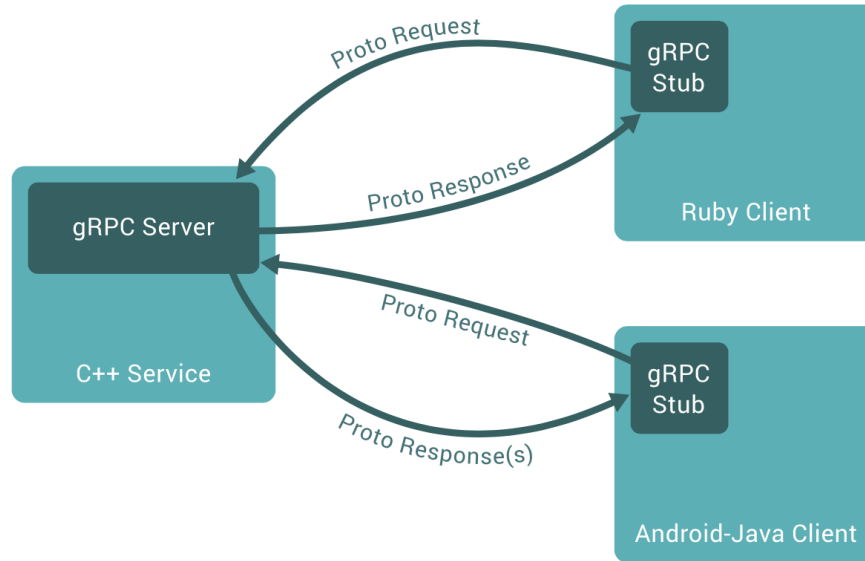




- Entwickelt von Google
- Plattform- und sprach-unabhängiger RPC
- Basiert auf protocol buffers (protobuf) für die sprachunabhängige Definition von Datentypen und Schnittstellen (neueste Version proto3)
- Verwendet als Transport HTTP/2
 - Verschiedene Performance-Optimierungen, u.a. nur eine Verbindung pro Client
 - HTTP1 öffnet ca. 4 – 8 Verbindungen für einen Client
 - Rückwärtskompatibel zu HTTP/1.1
- Sicherheit über SSL/TLS
- Open-Source seit 2015: <https://grpc.io>
- Verwendet in vielen Unternehmen, vor allem für Microservices

Übersicht

- Protocol Buffers bietet eine Interface Definition Language (IDL) und definiert die Schnittstellen und Nachrichten
- Unterstützt viele gängigen Sprachen : Java, Python, C#, C++, ...



Protocol Buffers - Messages

- Messages dienen der Serialisierung strukturierter Daten
- Der Inhalt wird in einer Text-Datei mit Endung **.proto** definiert:
 - Inhalt sind Name –Wert-Paare

```
message Person {  
    string name = 1;  
    int32 id = 2;  
    bool has_ponycopter = 3;  
}
```

- Mithilfe des protocol buffer compilers **protoc** können für die gewünschte Programmiersprache Zugriffsklassen generiert werden
 - Beinhalten getter- und setter-Methoden
 - Sowie Methoden zum De-/Serialisieren der Nachricht



Protocol Buffers - Service

- Als nächstes muss ein Service in einer **.proto**-Datei definiert werden
 - Dieser besteht aus Methoden, welche für die Typen der Parameter und dem optionalen Rückgabewert die protocol buffer messages verwenden
- **protoc** generiert client/server code (stub und skeleton)

```
service Greeter {  
    rpc SayHello (HelloRequest) returns (HelloReply) {}  
}  
  
message HelloRequest {  
    string name = 1;  
}  
  
message HelloReply {  
    string message = 1;  
}
```



Protocol Buffers – Service (2)

- Neben den „normalen“ Methoden-Aufrufen (ein Request und eine Response), gibt es auch die Möglichkeit Messages als Datenstrom zu übertragen
 - Der Datenstrom besteht aus einer Sequenz von Messages
 - Dies geht uni- oder bi-direktional
- Beispiel:

```
service Greeter {  
  rpc SayHello (HelloRequest) returns (HelloReply) {}  
  rpc LotsOfReplies (HelloRequest) returns (stream HelloReply) {}  
  rpc LotsOfGreetings (stream HelloRequest) returns (stream HelloReply) {}  
  rpc BidiHello (stream HelloRequest) returns (stream HelloReply) {}  
}
```



Weitere Eigenschaften

- Es gibt sowohl **synchrone** als auch **asynchrone** RPC-Aufrufe
- **Deadlines**: Client kann eine Frist angeben, bis wann ein RPC abgeschlossen sein muss
- **Terminierung**: Server und Client entscheiden lokal und unabhängig, wann ein RPC erfolgreich abgeschlossen wurde
- **Abbruch**: Server und Client können einen RPC-Aufruf jederzeit abbrechen
- **Metadata**: Sicherheits-Informationen (Authentisierung) für einen RPC-Aufruf

- Authentisierung pro Channel (Verbindung) oder für einen RPC-Aufruf, beispielweise mit SSL/TLS oder ALTS (für die Google Cloud Plattform)
- SSL: Secure Socket Layer
 - Verschlüsselte Verbindung
 - Letzte Version 3.0, 1996
- TLS: Transport Layer Security
 - Nachfolger von SSL
 - Basiert auf Zertifikaten, siehe letztes Kapitel
 - Neueste Version TLS 1.3, 2018

gRPC vs REST API

gRPC	REST
Protocol buffers – Binärformat (kompakt & schnell)	JSON – XML-basiert (größere Nachrichten & langsamer)
HTTP/2 (lower latency)	Ursprünglich nur HTTP/1.1 (higher latency); jetzt aber auch HTTP/2
Bidirektional & a/synchron	Nur Client-Server-Requests
Streaming möglich	Nur Request/Response
Freie API-Gestaltung möglich	Orientiert an HTTP-Requests (GET, PUT, PUT, ...)



5.5 Zusammenfassung



Zusammenfassung

- Fernaufrufe sind flexibel und viel einfacher zu verwenden, als Sockets
- Vorsicht ist bei Referenz-Parameter angesagt, falls diese automatisch serialisiert werden. Hier können u.U. sehr viele Daten übertragen werden was langsam ist.
- REST und gRPC sind weit verbreitet in der Praxis

